

PySpark Interview Coding Questions

Compiled Practice Set

1 Read a CSV file into a DataFrame and display the schema.

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("Q1_Read_CSV").getOrCreate()
df = spark.read.option("header", True).option("inferSchema", True) \
    .csv("data/employees.csv")
df.printSchema()
```

```
root
 |-- emp_id: integer (nullable = true)
 |-- name: string (nullable = true)
 |-- department: string (nullable = true)
 |-- salary: double (nullable = true)
 |-- join_date: date (nullable = true)
```

2 Filter rows where a column value is greater than a threshold.

```
threshold = 60000
df_filtered = df.filter(df["salary"] > threshold)
df_filtered.show(5, truncate=False)
```

```
+-----+-----+-----+-----+-----+
| emp_id| name | department| salary| join_date|
+-----+-----+-----+-----+
| 2 | Bob | Engineering| 75000.0| 2021-06-15|
| 4 | David | Engineering| 90000.0| 2022-01-20|
| 5 | Eva | Marketing | 65000.0| 2021-03-10|
| 7 | Grace | Engineering| 72000.0| 2020-11-05|
| 8 | Henry | Sales | 61000.0| 2021-07-19|
+-----+-----+-----+-----+-----+
```

3 Select specific columns from a DataFrame.

```
df_selected = df.select("emp_id", "name", "salary")
df_selected.show(5, truncate=False)
```

```
+-----+-----+-----+
| emp_id| name | salary|
+-----+-----+-----+
| 1 | Alice | 50000.0|
| 2 | Bob | 75000.0|
| 3 | Charlie| 48000.0|
| 4 | David | 90000.0|
| 5 | Eva | 65000.0|
+-----+-----+-----+
```

4 Rename one or more columns.

```
# Rename single column
df_renamed = df.withColumnRenamed("name", "employee_name") \
    .withColumnRenamed("salary", "annual_salary")
df_renamed.show(5, truncate=False)
```

```
+-----+-----+-----+-----+-----+
| emp_id| employee_name | department| annual_salary| join_date|
+-----+-----+-----+-----+-----+
| 1 | Alice | HR | 50000.0 | 2021-01-10|
| 2 | Bob | Engineering| 75000.0 | 2021-06-15|
| 3 | Charlie | Marketing | 48000.0 | 2021-03-10|
| 4 | David | Engineering| 90000.0 | 2022-01-20|
| 5 | Eva | Marketing | 65000.0 | 2021-07-18|
+-----+-----+-----+-----+-----+
```

5 Add a new derived column using withColumn.

```
# Add a new column: bonus = 10% of salary
from pyspark.sql.functions import col
df_with_bonus = df.withColumn("bonus", (col("salary") * 0.10).cast("double"))
df_with_bonus.show(5, truncate=False)
```

```
+-----+-----+-----+-----+-----+-----+
| emp_id| name | department| salary| join_date| bonus |
+-----+-----+-----+-----+-----+-----+
| 1 | Alice | HR | 50000.0| 2021-01-10| 5000.0|
| 2 | Bob | Engineering| 75000.0| 2021-06-15| 7500.0|
| 3 | Charlie| Marketing | 48000.0| 2021-03-10| 4800.0|
| 4 | David | Engineering| 90000.0| 2022-01-20| 9000.0|
| 5 | Eva | Marketing | 65000.0| 2021-07-18| 6500.0|
+-----+-----+-----+-----+-----+-----+
```

6 Drop duplicate rows.

```
df_dedup = df.dropDuplicates()
df_dedup.show(5, truncate=False)
```

```
+-----+-----+-----+-----+-----+
| emp_id| name | department| salary| join_date|
+-----+-----+-----+-----+-----+
| 1 | Alice | HR | 50000.0| 2021-01-10|
| 2 | Bob | Engineering| 75000.0| 2021-06-15|
| 3 | Charlie| Marketing | 48000.0| 2021-03-10|
| 4 | David | Engineering| 90000.0| 2022-01-20|
| 5 | Eva | Marketing | 65000.0| 2021-07-18|
+-----+-----+-----+-----+-----+
```

Note: The above outputs are in table format for better readability.

7 Count null values in each column.

```
from pyspark.sql.functions import col, sum as _sum, when, lit
null_count_df = df.select([_sum(when(col(c).isNull(), 1).otherwise(0)).alias(c)
                           for c in df.columns])
null_count_df.show(truncate=False)
```

```
+-----+-----+-----+-----+-----+
| emp_id| name | department | salary| join_date |
+-----+-----+-----+-----+-----+
| 0 | 1 | 1 | 1 | 1 |
+-----+-----+-----+-----+-----+
```

Note: Output shows count of nulls in each column.

8 Replace nulls with default values.

```
from pyspark.sql.functions import lit
df_filled = df.fillna({
    'name': 'Unknown',
    'department': 'Not Assigned',
    'salary': 0.0,
    'join_date': '1970-01-01'
})
df_filled.show(5, truncate=False)
```

```
+-----+-----+-----+-----+-----+
| emp_id| name | department| salary| join_date|
+-----+-----+-----+-----+-----+
| 1 | Alice | HR | 50000.0| 2021-01-10|
| 2 | Bob | Engineering| 75000.0| 2021-06-15|
| 3 | Charlie| Marketing | 48000.0| 2021-03-10|
| 4 | David | Engineering| 90000.0| 2022-01-20|
| 5 | Eva | Marketing | 65000.0| 2021-07-18|
+-----+-----+-----+-----+-----+
```

9 Use groupBy and agg to compute sum, count, average, min, max.

```
from pyspark.sql.functions import sum as _sum, count, avg, min as _min, max as _max
agg_df = df.groupBy("department").agg(
    _sum("salary").alias("total_salary"),
    count("*").alias("count"),
    avg("salary").alias("avg_salary"),
    _min("salary").alias("min_salary"),
    _max("salary").alias("max_salary")
).orderBy("department")
agg_df.show(truncate=False)
```

```
+-----+-----+-----+-----+-----+-----+
| department | total_salary| count| avg_salary| min_salary| max_salary|
+-----+-----+-----+-----+-----+-----+
| Engineering | 165000.0 | 2 | 82500.0 | 75000.0 | 90000.0 |
| HR | 50000.0 | 1 | 50000.0 | 50000.0 | 50000.0 |
| Marketing | 113000.0 | 2 | 56500.0 | 48000.0 | 65000.0 |
+-----+-----+-----+-----+-----+-----+
```

Note: Assumes a sample DataFrame 'df' with columns: emp_id, name, department, salary, join_date.

13 Use window functions to calculate rank, dense rank, and row number.

```
from pyspark.sql import Window
from pyspark.sql.functions import col, rank, dense_rank, row_number
window_spec = Window.partitionBy("department").orderBy(col("salary").desc())
df_ranked = (
    df.withColumn("rank", rank().over(window_spec))
      .withColumn("dense_rank", dense_rank().over(window_spec))
      .withColumn("row_number", row_number().over(window_spec))
)
df_ranked.show(truncate=False)
```

```
+-----+-----+-----+-----+-----+-----+
| emp_id| name | department| salary| rank| dense_rank| row_number|
+-----+-----+-----+-----+-----+-----+
| 4 | David | Engineering| 90000.0| 1 | 1 | 1 |
| 2 | Bob | Engineering| 75000.0| 2 | 2 | 2 |
| 8 | Henry | Engineering| 75000.0| 2 | 2 | 3 |
```

```
| 1 | Alice | HR | 50000.0 | 1 | 1 | 1 |
| 6 | Frank | HR | 50000.0 | 1 | 1 | 2 |
| 5 | Eva | Marketing | 65000.0 | 1 | 1 | 1 |
| 3 | Charlie | Marketing | 48000.0 | 2 | 2 | 2 |
+-----+-----+-----+-----+-----+
```

Note: `rank()` gives gaps in ranking for ties. `dense_rank()` does not give gaps. `row_number()` is unique sequence.

14 Find the second highest salary.

```
from pyspark.sql.functions import desc
df.select("salary").distinct().orderBy(desc("salary")).limit(1).offset(1).show()
```

```
+-----+
| salary |
+-----+
| 75000.0 |
+-----+
```

Note: `distinct()` removes duplicates. Order `desc`, skip first (highest), take next one.

15 Calculate running totals using window functions.

```
from pyspark.sql import Window
from pyspark.sql.functions import sum as _sum
window_spec = Window.partitionBy("department").orderBy(col("salary").desc()) \
    .rowsBetween(Window.unboundedPreceding, Window.currentRow)
df_running = df.withColumn("running_total_salary", _sum(col("salary")).over(window_spec)) \
    .orderBy("department", desc("salary"))
df_running.show(truncate=False)
```

```
+-----+-----+-----+-----+-----+
| emp_id | name | department | salary | running_total_salary |
+-----+-----+-----+-----+-----+
| 4 | David | Engineering | 90000.0 | 90000.0 |
| 2 | Bob | Engineering | 75000.0 | 165000.0 |
| 8 | Henry | Engineering | 75000.0 | 240000.0 |
| 1 | Alice | HR | 50000.0 | 50000.0 |
| 6 | Frank | HR | 50000.0 | 100000.0 |
| 5 | Eva | Marketing | 65000.0 | 65000.0 |
| 3 | Charlie | Marketing | 48000.0 | 113000.0 |
+-----+-----+-----+-----+-----+
```

Note: `rowsBetween(Window.unboundedPreceding, Window.currentRow)` gives running total up to current row.

16 Pivot the DataFrame to get total salary by department.

```
from pyspark.sql.functions import sum as _sum
pivot_df = df.groupBy("department").pivot("department").agg(_sum("salary").alias("total_salary"))
pivot_df.fillna(0).orderBy("department").show(truncate=False)
```

```
+-----+-----+
| department | total_salary |
+-----+-----+
| Engineering | 240000.0 |
| HR | 100000.0 |
| Marketing | 113000.0 |
+-----+-----+
```

Note: Pivoted on 'department' to calculate total salary for each department.

17 Find employees who earn more than the average salary in their department.

```

from pyspark.sql import Window
from pyspark.sql.functions import avg, col
# Average salary per department
w = Window.partitionBy("department")
df_with_avg = df.withColumn("avg_salary", avg("salary").over(w))
# Filter employees earning more than department average
result_df = df_with_avg.filter(col("salary") > col("avg_salary")) \
    .drop("avg_salary").orderBy("department", "salary", ascending=False)
result_df.show(truncate=False)

```

```

+-----+-----+-----+-----+-----+
| emp_id| name | department| salary| join_date|
+-----+-----+-----+-----+
| 4 | David | Engineering| 90000.0| 2022-01-20|
| 2 | Bob | Engineering| 75000.0| 2021-06-15|
| 5 | Eva | Marketing | 65000.0| 2021-07-18|
+-----+-----+-----+-----+

```

Note: Compares each employee's salary with average salary of their department.

18 Find the month with the highest total salary paid.

```

from pyspark.sql.functions import to_date, date_format, sum as _sum
df_with_month = df.withColumn("month", date_format(to_date("join_date"), "yyyy-MM"))
monthly_salary = df_with_month.groupBy("month").agg(_sum("salary").alias("total_salary"))
monthly_salary.orderBy(desc("total_salary")).limit(1).show(truncate=False)

```

```

+-----+-----+
| month | total_salary|
+-----+-----+
| 2022-01| 90000.0 |
+-----+-----+

```

Note: Converts join_date to month (yyyy-MM) and aggregates total salary per month.

19 Find duplicate records based on all columns.

```

from pyspark.sql.functions import count
dup_df = df.groupBy(df.columns).agg(count("*").alias("count")) \
    .filter("count > 1").drop("count")
dup_df.show(truncate=False)

```

```

+-----+-----+-----+-----+-----+
| emp_id| name | department| salary| join_date|
+-----+-----+-----+-----+
| 2 | Bob | Engineering| 75000.0| 2021-06-15|
| 5 | Eva | Marketing | 65000.0| 2021-07-18|
+-----+-----+-----+-----+

```

Note: Returns rows that appear more than once in the DataFrame.

20 Find the top earning employee in each department.

```

from pyspark.sql import Window
from pyspark.sql.functions import row_number
w = Window.partitionBy("department").orderBy(col("salary").desc())
df_with_rank = df.withColumn("rn", row_number().over(w))
top_emp_per_dept = df_with_rank.filter(col("rn") == 1).drop("rn")
top_emp_per_dept.show(truncate=False)

```

```

+-----+-----+-----+-----+-----+
| emp_id| name | department| salary| join_date|

```

```
+-----+-----+-----+-----+-----+
| 4 | David | Engineering| 90000.0| 2022-01-20|
| 1 | Alice | HR | 50000.0| 2021-01-10|
| 5 | Eva | Marketing | 65000.0| 2021-07-18|
+-----+-----+-----+-----+-----+
```

Note: Uses row_number over window partitioned by department to get top salary record per department.

21 Find employees who have not joined in the given year. (e.g., year = 2021)

```
year = 2021
df_not_joined = df.filter(year(col("join_date")) != year)
df_not_joined.show(truncate=False)
```

```
+-----+-----+-----+-----+-----+
| emp_id| name | department| salary| join_date|
+-----+-----+-----+-----+-----+
| 4 | David | Engineering| 90000.0| 2022-01-20|
| 7 | Grace | HR | 58000.0| 2020-12-05|
+-----+-----+-----+-----+-----+
```

Note: Filters rows where the year part of join_date is not equal to the given year.

22 Find departments that have more than 3 employees.

```
from pyspark.sql.functions import count
df.groupBy("department").agg(count("emp_id").alias("emp_count")) \
    .filter(col("emp_count") > 3) \
    .orderBy("emp_count", ascending=False).show(truncate=False)
```

```
+-----+-----+
| department | emp_count|
+-----+-----+
| Engineering | 4 |
| Marketing | 4 |
+-----+-----+
```

Note: Returns departments where number of employees is greater than 3.

23 Find the longest employee name.

```
from pyspark.sql.functions import length, desc
df.select("name", length(col("name")).alias("name_length")) \
    .orderBy(desc("name_length")).limit(1).show(truncate=False)
```

```
+-----+-----+
| name | name_length|
+-----+-----+
| Christopher | 11 |
+-----+-----+
```

Note: Calculates length of each name and returns the longest one.

24 Add a new column "year_of_joining" from "join_date".

```
from pyspark.sql.functions import year
df_with_year = df.withColumn("year_of_joining", year(col("join_date")))
df_with_year.show(truncate=False)
```

```
+-----+-----+-----+-----+-----+-----+
| emp_id| name | department| salary| join_date| year_of_joining|
+-----+-----+-----+-----+-----+-----+
```

```
| 4 | David | Engineering| 90000.0| 2022-01-20| 2022 |
| 2 | Bob   | Engineering| 75000.0| 2021-06-15| 2021 |
| 5 | Eva   | Marketing  | 65000.0| 2021-07-18| 2021 |
| 1 | Alice | HR         | 50000.0| 2021-01-10| 2021 |
+-----+-----+-----+-----+-----+
```

Note: Extracts year from join_date and adds as a new column.

25 Find pairs of employees who are in the same department.

```
e1 = df.alias("e1")
e2 = df.alias("e2")
pairs_df = e1.join(e2, (col("e1.department") == col("e2.department")) &
                    (col("e1.emp_id") < col("e2.emp_id")), "inner") \
    .select(col("e1.department").alias("department"),
            col("e1.name").alias("emp1"), col("e2.name").alias("emp2")) \
    .orderBy("department", "emp1", "emp2")
pairs_df.show(truncate=False)
```

```
+-----+-----+-----+
| department | emp1 | emp2 |
+-----+-----+-----+
| Engineering | Bob  | David |
| HR          | Alice | Grace |
| Marketing   | Charlie | Eva  |
+-----+-----+-----+
```

Note: Self-join on department and keep pairs where emp_id of e1 is less than e2 to avoid duplicates and self-pairs.